Alg2Code

Paul Molloy

November 2025

1 SuffixTreeAppl.java

```
package SuffixTreePackage;

/**
 * Class with methods for carrying out applications of suffix trees
 * @author David Manlove
 */

public class SuffixTreeAppl {

        /** The suffix tree */
        private SuffixTree t;

        /**
         * Default constructor.
         */
        public SuffixTreeAppl () {
                t = null;
        }

        /**
         * Constructor with parameter.
         *
         * @param tree the suffix tree
         */
        public SuffixTreeAppl (SuffixTree tree) {
                t = tree;
        }

        /**
         * Search the suffix tree t representing string s for a target x.
         * Stores -1 in Task1Info.pos if x is not a substring of s,
```

```
     * otherwise stores p in Task1Info.pos such that x occurs in s
     * starting at s[p] (p counts from 0)
     * - assumes that characters of s and x occupy positions 0 onwards
     *
     * @param x the target string to search for
     *
     * @return a Task1Info object
     */
    public Task1Info searchSuffixTree(byte[] x) {
    if (t == null || x == null || x.length == 0) {
        return new Task1Info(-1, null);
    }

    SuffixTreeNode currentNode = t.getRoot();
    int index = 0;

    while (index < x.length) {
        SuffixTreeNode nextNode = t.searchList(currentNode.getChild(), x[ind
        if (nextNode == null) {
            return new Task1Info(-1, null);
        }

        int edgeStart = nextNode.getLeftLabel();
        int edgeEnd = nextNode.getRightLabel();
        byte[] treeString = t.getString();

        for (int i = edgeStart; i <= edgeEnd && index < x.length; i++, index
            if (x[index] != treeString[i]) {
                return new Task1Info(-1, null);
            }
        }

        currentNode = nextNode;
            }
            return new Task1Info(currentNode.getSuffix(), currentNode);
}

    /**
     * Search suffix tree t representing string s for all occurrences of tar
     * Stores in Task2Info.positions a linked list of all such occurrences.
     * Each occurrence is specified by a starting position index in s
     * (as in searchSuffixTree above).  The linked list is empty if there
     * are no occurrences of x in s.
     * - assumes that characters of s and x occupy positions 0 onwards
     *
     * @param x the target string to search for
```

```
     *
     * @return a Task2Info object
     */
    public Task2Info allOccurrences(byte[] x) {

            SuffixTreeAppl suffixTreeAppl = new SuffixTreeAppl(t);
            Task1Info task1 = suffixTreeAppl.searchSuffixTree(x);

            SuffixTreeNode start = task1.getMatchNode();

            Task2Info task2 = new Task2Info();
            if (start == null) {
                    return task2;
            } else if (start.getSuffix() == -1) {
                    task2 = recLeafCount(start.getChild(), task2);
            } else {
                    task2.addEntry(start.getSuffix());
            }
            return task2;
    }

    public Task2Info recLeafCount(SuffixTreeNode current, Task2Info task2) {
            Task2Info empty = new Task2Info();
            if (current == null) {
                    return empty;
            } else if (current.getSuffix() == -1 ) {
                    recLeafCount(current.getChild(), task2);
            } else {
                    task2.addEntry(current.getSuffix());
            }
            recLeafCount(current.getSibling(), task2);
            return task2;
    }

/**
 * Traverses suffix tree t representing string s and stores ln, p1 and
 * p2 in Task3Info.len, Task3Info.pos1 and Task3Info.pos2 respectively,
 * so that s[p1..p1+ln-1] = s[p2..p2+ln-1], with ln maximal;
 * i.e., finds two embeddings of a longest repeated substring of s
 * - assumes that characters of s occupy positions 0 onwards
 * so that p1 and p2 count from 0
 *
 * @return a Task3Info object
 */
public Task3Info traverseForLrs() {
    if (t == null) {
```

```
        return new Task3Info(0, -1, -1);
    }

    Task3Info lrsInfo = new Task3Info(0, -1, -1);
    findLrs(t.getRoot(), 0, lrsInfo);
    return lrsInfo;
}

/**
 * Helper method to recursively find the longest repeated substring.
 *
 * @param node the current node
 * @param depth the current depth in the tree
 * @param lrsInfo the Task3Info object to store the result
 */
private void findLrs(SuffixTreeNode node, int depth, Task3Info lrsInfo) {
    if (node == null) {
        return;
    }

    if (node.getChild() != null) {
        if (depth > lrsInfo.getLen()) {
            lrsInfo.setLen(depth);
            lrsInfo.setPos1(node.getChild().getSuffix());
            lrsInfo.setPos2(node.getChild().getSibling() != null ? node.getC
        }
    }

    SuffixTreeNode child = node.getChild();
    while (child != null) {
        findLrs(child, depth + (child.getRightLabel() - child.getLeftLabel()
        child = child.getSibling();
    }
}

/**
     * Traverse generalised suffix tree t representing strings s1 (of length
     * s1Length), and s2, and store ln, p1 and p2 in Task4Info.len,
     * Task4Info.pos1 and Task4Info.pos2 respectively, so that
     * s1[p1..p1+ln-1] = s2[p2..p2+ln-1], with len maximal;
     * i.e., finds embeddings in s1 and s2 of a longest common substring
     * of s1 and s2
     * - assumes that characters of s1 and s2 occupy positions 0 onwards
     * so that p1 and p2 count from 0
     *
     * @param s1Length the length of s1
```

```
         *
         * @return a Task4Info object
         */
        public Task4Info traverseForLcs(int s1Length) {
    Task4Info task4 = new Task4Info();
    SuffixTreeNode root = t.getRoot();

    if (root == null || root.getChild() == null) {
        return task4;
    }

    recFindLcs(task4, root, 0, s1Length);
    return task4;
}

    public Task4Info recFindLcs(Task4Info task4, SuffixTreeNode current, int dep
        if (current == null) {
            return task4;
        }

        SuffixTreeNode child = current.getChild();

        if (child.getSuffix() != -1 && child.getSibling() == null) {
            return task4;
        }

        if (child.getSuffix() == -1 && child.getSibling() == null) {
            depth += current.getRightLabel() - current.getLeftLabel() + 1;
            task4 = recFindLcs(task4, child, depth, splitPoint);
        } else {
            depth += current.getRightLabel() - current.getLeftLabel() + 1;
            task4.setString1Leaf(false);
            task4.setString2Leaf(false);
            int s1 = 0;
            int s2 = 0;

            while (child != null) {
                if (child.getSuffix() == -1) {
                    task4 = recFindLcs(task4, child, depth, splitPoint);
                }

                if (child.getLeftLabel() <= splitPoint) {
                    task4.setString1Leaf(true);
                    s1 = child.getLeftLabel();
                } else {
                    task4.setString2Leaf(true);
```

```
                    s2 = child.getLeftLabel();
                }

                child = child.getSibling();
            }

            if (task4.getString1Leaf() && task4.getString2Leaf() && task4.getLen
                task4.setLen(depth - 1);
                task4.setPos1(s1 - task4.getLen());
                task4.setPos2(s2 - task4.getLen() - splitPoint);
            }
        }

        return task4;
    }
}
```

2 main.java

```
import SuffixTreePackage.*;

/**
 * Main class - for accessing suffix tree applications
 *
 * @author
 */

public class Main {

    /**
     * The main method.
     *
     * @param args the arguments
     */
    public static void main(String args[]) {
        String errorMessage = "Required syntax:\n";
        errorMessage += "  java Main SearchOne <filename> <query string> for Tas
        errorMessage += "  java Main SearchAll <filename> <query string> for Tas
        errorMessage += "  java Main LRS <filename> for Task 3\n";
        errorMessage += "  java Main LCS <filename1> <filename2> for Task 4";

        if (args.length < 2) {
            System.out.println(errorMessage);
        } else {
```

```
String command = args[0];

switch (command) {
    case "SearchOne": {
        if (args.length < 3) {
            System.out.println(errorMessage);
            break;
        }

        String filename = args[1];
        String queryString = args[2];

        try {
            FileInput fileInput = new FileInput(filename);
            byte[] fileContent = fileInput.readFile();

            SuffixTree tree = new SuffixTree(fileContent);
            SuffixTreeAppl treeAppl = new SuffixTreeAppl(tree);

            Task1Info result = treeAppl.searchSuffixTree(queryString

            if (result.getPos() == -1) {
                System.out.println("The string" + queryString + " no
            } else {
                System.out.println("The string " + queryString + " w
                        + " in file " + filename);
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
        break;
    }
    case "SearchAll": {
        if (args.length < 3) {
            System.out.println(errorMessage);
            break;
        }

        String filename = args[1];
        String queryString = args[2];

        try {
            FileInput fileInput = new FileInput(filename);
            byte[] fileContent = fileInput.readFile();

            SuffixTree tree = new SuffixTree(fileContent);
```

```
            SuffixTreeAppl treeAppl = new SuffixTreeAppl(tree);

            Task2Info result = treeAppl.allOccurrences(queryString.g

            if (result.getPositions().isEmpty()) {
                System.out.println("The string" + queryString + " no
                System.out.println("Query string not found in the fi
            } else {
                System.out.println("The string " + queryString + " w

                for (Integer pos : result.getPositions()) {
                    System.out.println(pos);
                }

                System.out.println("The total number of occurences i
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
        break;
    }
    case "LRS": {
        if (args.length < 2) {
            System.out.println(errorMessage);
            break;
        }

        String filename = args[1];

        try {
            FileInput fileInput = new FileInput(filename);
            byte[] fileContent = fileInput.readFile();

            SuffixTree tree = new SuffixTree(fileContent);
            SuffixTreeAppl treeAppl = new SuffixTreeAppl(tree);

            Task3Info result = treeAppl.traverseForLrs();

            if (result.getLen() == 0) {
                System.out.println("No repeated substring found.");
            } else {
                System.out.println("An LRS in " + filename + "is of
                System.out.println("Starting position of one occuren
                System.out.println("Starting position of another occ
            }
        } catch (Exception e) {
```

```
                System.out.println("Error: " + e.getMessage());
            }
            break;
        }
        case "LCS": {
            if (args.length < 3) {
                System.out.println(errorMessage);
                break;
            }

            String filename1 = args[1];
            String filename2 = args[2];

            try {
                FileInput fileInput1 = new FileInput(filename1);
                FileInput fileInput2 = new FileInput(filename2);
                byte[] fileContent1 = fileInput1.readFile();
                byte[] fileContent2 = fileInput2.readFile();

                SuffixTree tree = new SuffixTree(fileContent1, fileConte
                SuffixTreeAppl treeAppl = new SuffixTreeAppl(tree);

                Task4Info result = treeAppl.traverseForLcs(fileContent1.

                if (result.getLen() == 0) {
                    System.out.println("No common substring found.");
                } else {
                    System.out.println("An LCS of " + filename1 + " and
                            + result.getLen());
                    System.out.println("Starting position in " + filenam
                    System.out.println("Starting position in " + filenam
                }
            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
            break;
        }
        default:
            System.out.println(errorMessage);
    }
  }
 }
}
```

3 SuffixTree.java

```
    package SuffixTreePackage;

/**
 * Class for construction and manipulation of suffix trees based on a list
 * of children at each node.
 *
 * Includes naive O(n^2) suffix tree construction algorithm based on
 * repeated insertion of suffixes and edge-splitting.
 *
 * @author Ela Hunt, Rob Irving, David Manlove
 */

public class SuffixTree {

        /** Root node of the suffix tree. */
        private SuffixTreeNode root;

        /** String (byte array) corresponding to suffix tree. */
        private byte [] s;

        /** Length of string corresponding to suffix tree (without termination c
        private int stringLen;

        /**
         * Builds the suffix tree for a given string.
         *
         * @param sInput the string whose suffix tree is to be built
         * - assumes that '$' does not occur as a character anywhere in sInput
         * - assumes that characters of sInput occupy positions 0 onwards
         */
        public SuffixTree (byte [] sInput) {
                root = new SuffixTreeNode(null, null, 0, 0, -1);
// create root node of suffix tree;
                stringLen = sInput.length;
                s = new byte[stringLen + 1]; // create longer byte array ready f
                System.arraycopy(sInput, 0, s, 0, stringLen);
                s[stringLen] = (byte) '$';   // append termination character to
                buildSuffixTree();           // build the suffix tree
        }

        /**
         * Builds a generalised suffix tree for two given strings.
         *
```

```
 * @param sInput1 the first string
 * @param sInput2 the second string
 * - assumes that '$' and '#' do not occur as a character anywhere in sI
 * - assumes that characters of sInput1 and sInput2 occupy positions 0 o
 */
public SuffixTree (byte[] sInput1, byte[] sInput2) {
root = new SuffixTreeNode(null, null, 0, 0, -1);
        int len1 = sInput1.length;
        int len2 = sInput2.length;
        stringLen = len1+len2+1;
        s = new byte [stringLen+1];
        System.arraycopy(sInput1, 0, s, 0, len1);
        s[len1] = (byte) '#';
        System.arraycopy(sInput2, 0, s, len1+1, len2);
        s[stringLen] = (byte) '$';
        buildSuffixTree();
}

/**
 * Builds the suffix tree.
 */
private void buildSuffixTree() {
        try {
                for (int i=0; i<= stringLen; i++) {
                        // for large files, the following line may be us
                        // indicating the progress of the suffix tree co
                        // if (i % 10000==0) System.out.println(i);

                        // raise an exception if the text file contained
                        if (s[i] == (byte) '$' && i < stringLen)
                                throw new Exception();
                        else
                                insert(i);  // insert suffix number i of
                }
        }
        catch (Exception e) {
                System.out.println("Text file contains a $ character!");
                System.exit(-1);
        }
}

/**
 * Given node nodeIn of suffix tree and character ch, search nodeIn,
 * plus all sibling nodes of nodeIn, looking for a node whose left
 * label x satisfies ch == s[x].
 * - Assumes that characters of s occupy positions 0 onwards
```

```
     *
     * @param nodeIn a node of the suffix tree
     * @param ch the character to match
     *
     * @return the matching suffix tree node (null if none exists)
     */
    public SuffixTreeNode searchList (SuffixTreeNode nodeIn, byte ch) {

            SuffixTreeNode next = nodeIn;
            SuffixTreeNode nodeOut = null;

            while (next != null) {
                    if (next.getLeftLabel() < stringLen && s[next.getLeftLab
                    {
                            nodeOut = next;
                            next = null;
                    }
                    else
                            next = next.getSibling();
            }
            return nodeOut;  // return matching node if successful, or null
    }

    /**
     * Inserts suffix number i of s into suffix tree.
  * - assumes that characters of s occupy positions 0 onwards
     *
     * @param i the suffix number of s to insert
     */
    private void insert(int i) {

            int pos, j, k;
            SuffixTreeNode current, next;
            pos = i;  // position in s
            current = root;

            while (true) {
                    // search for child of current with left label x such th
                    next = searchList(current.getChild(), s[pos]);

                    if (next == null) {
                            // current node has no such child, so add new on
                            // positions pos onwards of s
                            current.addChild(pos, stringLen, i);
                            break;
                    }
```

```
                else {
                        // try to match s[node.getLeftLabel()+1..node.ge
                        // segment of s starting at position pos+1
                        j = next.getLeftLabel() + 1;
                        k = pos + 1;

                        while (j <= next.getRightLabel()) {
                                if (s[j] == s[k]) {
                                        j++;
                                        k++;
                                }
                                else
                                        break;
                        }
                        if (j > next.getRightLabel()) {
                                // succeeded in matching whole segment,
                                pos = k;
                                current = next;
                        }
                        else {
                                /* succeeded in matching s[next.getLeftL
                                 * s[pos..k-1]. Split the node next so
                                 * now j-1. Create two children of next
                                 * suffix i, with left label k and right
                                 * and (2) with left label j and right l
                                 * whose children are those of next (if
                                 * number is equal to that of next. */

                                SuffixTreeNode n1 = new SuffixTreeNode(n
                                SuffixTreeNode n2 = new SuffixTreeNode(n

            // now update next's right label, list of children and suffi
                                next.setRightLabel(j-1);
                                next.setChild(n2);
                                next.setSuffix(-1); // next is now an in
                                break;
                        }
                }
        }
}

/**
 * Gets the root node.
 *
 * @return the root node
 */
```

```
        public SuffixTreeNode getRoot() { return root; }

        /**
         * Sets the root node.
         *
         * @param node the new root node
         */
        public void setRoot(SuffixTreeNode node) { root = node; }

        /**
         * Gets the string represented by the suffix tree.
         *
         * @return the string represented by the suffix tree
         */
        public byte[] getString() { return s; }

        /**
         * Sets the string represented by the suffix tree.
         *
         * @param sInput the new string represented by the suffix tree
         */
        public void setString(byte [] sInput) { s = sInput; }

        /**
         * Gets the length of the string represented by the suffix tree.
         *
         * @return the length of the string represented by the suffix tree
         */
        public int getStringLen() { return stringLen; }

        /**
         * Sets the length of the string represented by the suffix tree.
         *
         * @param len the new length of the string represented by the suffix tre
         */
        public void setStringLen(int len) { stringLen = len; }
}
```